

به نام خدا

گزارش فاز اول پروژه مبانی هوش مصنوعی

امیرحسین صبوری خسروشاهی

40231034



فضای حالت

در بازی سوکوبان با توجه به موجودیت هایی که داریم (1 بازیکن 2 جعبه 3 سم 4 سیب 5 دیوار 6 هدف 7 تایمر 8 سلول ها و ...

باید بفهمیم که تمایز دو حالت بر اساس چه پارامتر هایی خواهد بود و اینکه فضای حالت باید چگونه در نظر گرفته شود تا از حالات اضافی و تکراری پرهیز شود

با توجه به اینکه بازیکن ما میتواند در هر سلولی قرار بگیرد اما مثلا در جاهایی که دیوار قرار دارد یا خود جعبه قرار دارد نمیتواند قرار بگیرد (در سلول های قابل دسترس میتواند قرار بگیرد)

جعبه ها نیز هر چند تا که باشند می توانند به تعداد ترکیبشان با کل سلول های خالی میتوانند موقعیت داشته باشند

ما موقعیت دیوار ها و موقعیت خانه های هدف و موقعیت اولیه تمام سیب ها و موقعیت اولیه تمام سم ها در فضای حالت برایمان مهم نیست چون همواره ثابت هستند و تغییر نمیکند تنها موجودیت هایی که موقعیت ان ها را دقیق ذخیره باید کرد بازیکن و دیوار هست چون موقعیتشان ممکن هست در طی زمان تفاوت بکند .

درمورد سیب ها و سم ها ما فقط موقعیت های سیب ها و سم های خورده نشده برایمان مهم هست و به جای اینکه در هر لحظه موقعیت همه سیب ها و سم ها را که چه خورده شده اند چه خورده نشده اند را نگه داریم ما فقط موقعیت های سیب و سم هایی که خورده نشده اند را نگه می داریم چون فقط ان ها مهم هستند و از حالات تکراری جلوگیری می شود .

تایمر سیب نیز برایمان اهمیت دارد زیرا باید بدانیم که تا چه زمانی باید اثر سیب را همچنان اعمال بکنیم و چه زمانی دیگر نیاز نیست اعمال بکنیم. تایمر سیب با توجه به اینکه از 0 تا 10 میباشد دارای 11 حالت مختلف هست

در مورد سم بحث تایمر نداریم چون اثر سم برخلاف سیب ها دایمی هست و 2 حالت دارد یا فعال هست یا غیرفعال هست

وضعیت سیب ها دو حالت

اگر تعداد جعبه هایمان N_B باشد

اگر سلول های قابل دسترس را F در نظر بگیریم

اگر N_A تعداد سیب های اولیه باشد

اگر N_R تعداد سم های اولیه باشد

تخمین کران بالایی از اندازه فضای حالت به شکل زیر هست:

پیاده سازی تابع Successors

تابع successor ما در کلاس SokobanProblem قرار دارد و این تابع تمام حرکت های قانونی و ممکن رو از حالت فعلی را مشخص میکند . و برای هر حرکت ما هزینه انتقال را برمیگرداند

تابع successor ما یک تابعی هست که برای هر چهارجهت حرکت حالت جانشین را بررسی و در صورت مجاز بودنش yield میکند اگر مجاز نباشد مثلا دیوار باشد یا پشت جعبه ای که می خواهیم هل بدهیم جعبه دیگری باشد یا پشت جعبه ای که می خواهیم هل بدهیم دیوار باشد برمیگردد و اجازه بسط نمی دهد .

حالت الف) حرکت ما ساده باشد

اگر حرکت ما ساده باشد در ابتدا تابع ما تایمر سیب را یک واحد می کاهد و اگر مجاز باشد دو تا شرط را چک میکند (1) اینکه آیا در موقعیت جدید سیب وجود دارد یا نه . اگر موجود بود ان سیب را از لیست سیب ها حذف کرده و تایمر را به 10 ریست میکند . اگر موقعیت بعدی ما سم باشد ان را از لیست سم ها حذف کرده و Boolean مربوط به سم را به True تغییر می دهیم . بعد یک new_state از تایپ State ایجاد می کنیم و تغییرات جدید را به ان اعمال میکنیم .

و اگر در موقعیت بعدی نه سیب بود نه سم ما فقط تایمر سیب را کم میکنیم و np جدید را اپدیت میکنیم در نهایت بن بست بودن را چک میکنیم اگر بن بست نداشت cost را حساب کرده و yield میکنیم

حالت ب) موقعیت بعدی جعبه داشته باشد

در این حالت موقعیت بعدی ما جعبه ای وجود دارد در ابتدا np مربوط به موقعیت بعدی همان after را اپدیت می کنیم بعد در ابتدا چک می کنیم که آیا پشت جعبه دیوار یا جعبه دیگری حضور دارد یا نه؟ اگر نبود در مرحله بعد چک می کنیم که آیا حالت بن بست پیش می آید یا نه ؟ اگر نیامد وارد if میشویم و موقعیت قبلی جعبه را حذف کرده و موقعیت جدید جعبه را اضافه میکنیم اگر موقعیت جدید سیب داشت از لیست سیب ها ان را حذف میکنیم چنانچه سم داشت از لیست سم ها حذف میکنیم . در نهایت new_state مان را تشکیل داده و فیلد های ان را با داده های جدید مقدار دهی می کنیم. بعد دوباره بن بست را بررسی میکنیم اگر نبود هزینه را محاسبه میکنیم و برمیگردانیم

کد پیاده سازی تابع پسین:

```
def successors(self, s: State):
    px, py = s.player
    boxes = set(s.boxes)
    apples = set(s.apples)
    poisons = set(s.poisons)
    apple_timer = s.apple_timer
```

```

        poison_active = s.poison_active

    for a in ACTIONS:
        dx, dy = DIRS[a]
        np = (px + dx, py + dy)

        new_apple_timer = max(0, apple_timer - 1) if apple_timer >
0 else 0

        if self.is_free(s, np):
            new_apples = set(apples)
            new_poisons = set(poisons)
            new_poison_active = poison_active

            if np in new_apples:
                new_apples.remove(np)
                new_apple_timer = 10

            if np in new_poisons:
                new_poisons.remove(np)
                new_poison_active = True

            new_state = State(
                player=np,
                boxes=frozenset(boxes),
                apples=frozenset(new_apples),
                poisons=frozenset(new_poisons),
                apple_timer=new_apple_timer,
                poison_active=new_poison_active
            )

            if self.use_deadlocks and self.is_deadlock(new_state):
                continue

            cost = self.cost(s, a, new_state)
            yield a, new_state, cost
            continue

    if np in boxes:

```

```

        after = (np[0] + dx, np[1] + dy)
        if self.in_bounds(after) and (after not in self.walls)
and (after not in boxes):
            # استفاده میکنیم اینجا از _is_simple_corner_deadlock
            if self.use_deadlocks and
self._is_simple_corner_deadlock(after):
                continue

            new_boxes = set(boxes)
            new_boxes.remove(np)
            new_boxes.add(after)

            new_apples = set(apples)
            new_poisons = set(poisons)
            new_poison_active = poison_active

            if after in new_apples:
                new_apples.remove(after)
            if after in new_poisons:
                new_poisons.remove(after)

            new_state = State(
                player=np,
                boxes=frozenset(new_boxes),
                apples=frozenset(new_apples),
                poisons=frozenset(new_poisons),
                apple_timer=new_apple_timer,
                poison_active=new_poison_active
            )

            if self.use_deadlocks and
self.is_deadlock(new_state):
                continue

            cost = self.cost(s, a, new_state)
            yield a, new_state, cost

```

پیاده سازی تابع Cost()

من در پیاده سازی تابع هزینه طبق گفته های متن فایل پروژه به صورت زیر عمل کردم:

✓ هزینه های پایه :

(1) حرکت ساده : 1

(2) هل دادن : 5

✓ تخفیف مربوط به سیب:

(1) حرکت ساده: همان 1 باقی میماند

(2) هل دادن : از 5 واحد به 1 واحد کاهش پیدا میکند

✓ جریمه مربوط به سم :

(1) اگر سم فعال باشد : تمامی حرکات 3 واحد به هزینه شان اضافه

میشود

سناریو های ترکیبی نیز با کدی که نوشته ام درست جواب میگیرند :

حرکت ساده با سیب فعال و سم فعال : $1 + 0 + 3 = 4$

هل دادن با سیب فعال و سم فعال : $5 - 4 + 3 = 4$

هل دادن با سیب غیرفعال و سم فعال : $5 + 0 + 3 = 8$

ترتیب اجرای قوانین: (1) تعیین نوع حرکت و هزینه پایه (2) اعمال تخفیف سیب در صورت فعال بودن (3) اعمال جریمه سم و افزودن جریمه

کد پیاده سازی هزینه:

```
def cost(self, s: State, a: str, sp: State) -> float:
    is_pushing = s.bboxes != sp.bboxes
    base_cost = 5.0 if is_pushing else 1.0

    if s.apple_timer > 0 and is_pushing:
        base_cost = 1.0

    if s.poison_active:
        base_cost += 3.0

    return base_cost
```

تابع Deadlock()

تابع بن بست در خود سه تا از حالات وقوع بن بست را چک میکند و به محض اینکه یکی از آن ها True شد دیگر گره مربوط به آن را بررسی نمی کند تابع is_deadlock این کار را انجام می دهد در واقع به ازای هر box بررسی میکند که آیا box درون حالت هدف قرار دارد یا ندارد اگر قرار داشت که خیلی خوب هست و ما در حالت بن بست قرار نخواهیم گرفت در غیر اینصورت خروجی سه تا تابع بررسی میشود .

کد مربوط به is_deadlock()

```
def is_deadlock(self, state: State) -> bool:
    """
    Sokoban تشخیص بنبستهای مختلف در بازی
    """
    for box in state.boxes:
        if box in self.goals:
            continue # جعبهای که روی هدف است بنبست نیست

        # بنبست گوشه کلاسیک
        if self._is_simple_corner_deadlock(box):
            return True

        # بنبست بین دو دیوار
        if self._is_wall_deadlock(box, state.boxes):
            return True

        # بنبست لبه (برای نقشههای بزرگتر)
        if self._is_edge_deadlock(box):
            return True

    return False
```

1) تابع بن بست گوشه ساده:

در این تابع بررسی می کنیم که آیا جعبه در حالت گوشه قرار دارد یا نه اگر قرار داشت که بلا فاصله True برمیگردانیم . این کار را با بررسی خانه های up,down,left,right انجام میدهیم اگر هر دو تای مجاور هم دیوار بودند انگاه به بن بست رسیده ایم



```
def _is_simple_corner_deadlock(self, box: Tuple[int, int]) -> bool:
    """بنبست گوشه: جعبه در گوشه بین دو دیوار"""
    if box in self.goals:
        return False

    x, y = box

    # بررسی آیا دیوارها در گوشه هستند
    up = (x, y-1) in self.walls
    down = (x, y+1) in self.walls
    left = (x-1, y) in self.walls
    right = (x+1, y) in self.walls

    return (up and left) or (up and right) or (down and left) or
    (down and right)
```

2) تابع بن بست بین دیوارها/جعبه ها

در این تابع بررسی میکنیم تا از بن بست ای که در آن جعبه بین دو دیوار افقی و دو دیوار عمودی گیر افتاده و در جلو به هدف هم نمیرسد جلوگیری کنیم . در اینجا این قابلیت را گذاشته ایم که اگر یک خانه جلوتر جعبه هم هدف بود بن بست نگیریم ولی اگر در خانه های جلویی هدفی نبود ان را بن بست در نظر می گیریم.



•

پیاده سازی به شکل کد:

```
def _is_wall_deadlock(self, box: Tuple[int, int], boxes:
FrozenSet[Tuple[int, int]]) -> bool:

    # اگر جعبه روی هدف باشد، بنیست نیست. 1.
    if box in self.goals:
        return False

    x, y = box

    left_blocked = (x-1, y) in self.walls or (x-1, y) in boxes
    right_blocked = (x+1, y) in self.walls or (x+1, y) in boxes

    # اگر جعبه افقی مسدود باشد
    if left_blocked and right_blocked:
        # اگر در حالت گیر کردن افقی، هیچ هدف قابل دسترسی در بالا یا پایین نباشد، بنیست است
        up_is_goal = (x, y-1) in self.goals
        down_is_goal = (x, y+1) in self.goals

        # اگر در هر دو جهت باز (بالا و پایین) هدف وجود نداشته باشد، بنیست قطعی است
        # (بود corner deadlock فرض میشود خانههای بالا و پایین دیوار نیستند، در غیر این صورت)
        if not up_is_goal and not down_is_goal:
            return True

    # اگر هدف وجود دارد (مثلاً هدف در پایین است)، اما حرکت به آن مسدود باشد، همچنان بنیست نیست
```

نمیشود `True` در مثال تصویری شما، خانه پایین هدف است و مسدود نیست، بنابراین این حالت

```
# --- (Vertical Blocking) --- بررسی مسدود شدن در محور عمودی ---
up_blocked = (x, y-1) in self.walls or (x, y-1) in boxes
down_blocked = (x, y+1) in self.walls or (x, y+1) in boxes

# اگر جعبه عمودی مسدود باشد
if up_blocked and down_blocked:
    # اگر در حالت گیر کردن عمودی، هیچ هدف قابل دسترسی در چپ یا راست نباشد، بنبست است

    left_is_goal = (x-1, y) in self.goals
    right_is_goal = (x+1, y) in self.goals

    if not left_is_goal and not right_is_goal:
        return True

return False
```

in

3) تابع بن بست لبه

در این حالت اگر جعبه به مسیر لبه ها و اضلاع بیرونی `map` بیفتد و هیچ هدفی ان جا نباشد عملا در یک بن بست می افتد و دیگر نمی تواند از ان بیرون برود چون بازیکن قدرت کشیدن را ندارد و فقط می تواند هل بدهد

اما بررسی این که در لبه مد نظر نقشه در خانه هایش ایا هدف هست یا نیست تحلیل را پیچیده میکند پس به همین اکتفا میکنیم که اگر جعبه در لبه ای قرار گرفت و دقیقا روی هدف قرار گرفت بن بست اعلام نکنیم در غیر این صورت بن بست خواهیم داشت.



پیاده سازی به شکل کد :

```
def _is_edge_deadlock(self, box: Tuple[int, int]) -> bool:
```

```

"""بنیست لبه: جعبه در لبه نقشه گیر کرده"""
if box in self.goals:
    return False
x, y = box

# اگر جعبه در لبه نقشه باشد و دیوار کناری داشته باشد
if x == 0 or x == self.w - 1 or y == 0 or y == self.h - 1:
    # بررسی جهت‌های ممکن برای حرکت جعبه
    if x == 0 and (1, y) in self.walls: # لبه چپ
        return True
    if x == self.w - 1 and (self.w - 2, y) in self.walls: # لبه
        return True
    if y == 0 and (x, 1) in self.walls: # لبه بالا
        return True
    if y == self.h - 1 and (x, self.h - 2) in self.walls: # لبه
        return True

return False

```

راست

پایین

پیاده سازی الگوریتم DFS :

الگوریتم DFS دارای ویژگی هایی مثل استفاده از پشته به عنوان fringe هست ابتدا عمیق ترین گره را گسترش می دهیم و پیاده سازی اش به شکل یک fringe به شکل FIFO هست الگوریتم DFS در صورت محدودیت عمق کامل هست ولی تضمینی به پیدا کردن جواب بهینه نیست و سمت چپ ترین جواب را در درخت پیدا می کند و به شرط نبود دور و محدود بودن درخت کامل هست حافظه اش به نسبت کم ولی از لحاظ زمان ممکن هست که مجبور شود کل گره های درخت را در بدترین حالت جست و جو کند

من در کد آمده ام و یک max_depth در نظر گرفته ام که خودم در دستم باشد که dfs تا کدام عمق جلو برود و بررسی می کند و طبق آزمایشی که انجام دادم در بازی ما جواب را پیدا میکند ولی اصلا بهینه نیست و خیلی زمانبر هست و طول می کشد جواب غیر بهینه هست من در کد از stack استفاده کردم به خاطر منطق LIFO ای که DFS دارا هست . یک مجموعه visited در نظر گرفته ام که گره های بازدید شده را بررسی بکند .

در هر state بررسی میکند آیا به هدف رسیده ایم اگر اره path را برمیگرداند اگر state مان قبل ویزیت شده باشد و یا عمقش بیش از max_depth باشد بررسی نمیکند و مابقی کد را اجرا نمیکند در غیر اینصورت بعد بررسی شرایط بن بست state را به مجموعه visited اضافه می کند چک می کند که در شرایط بن بست قرار نگیریم

در ادامه کد تمام وضعیت های ممکن بعدی (جانشینان) را از وضعیت فعلی پیدا کرده و ان ها را به پشته DFS اضافه میکند تا بعدا مورد بررسی قرار بگیرند. فقط وضعیت هایی که قبلا بازدید نشده اند را به stack اضافه می کنیم.

اگر هدف پیدا نشد می نویسد 2051: None expanded به طور مثال

کد پیاده سازی DFS :

```
def dfs_search(problem: SokobanProblem, max_depth: int = 1000):
    start = problem.initial
    stack = [(start, [], 0)] # (state, path, depth)
    visited = set() # برای نگهداری وضعیتهای بازدید شده

    while stack:
        state, path, depth = stack.pop()

        # اگر به هدف رسیدیم
        if problem.is_goal(state):
            return path, {"expanded": len(visited)}

        # جلوگیری از بازدید دوباره از وضعیتها یا عبور از عمق بیشتر از max_depth
        if state in visited or depth >= max_depth:
            continue

        # بررسی بنبستها قبل از ادامه
        if problem.use_deadlocks and problem.is_deadlock(state):
            continue

        visited.add(state)

        for action, next_state, cost in problem.successors(state):
            # اضافه میکنیم stack فقط وضعیتهایی که قبلاً بازدید نشدهاند را به
            if next_state not in visited:
                stack.append((next_state, path + [action], depth + 1))

    return None, {"expanded": len(visited)}
```

نتایج برگردانده شده از level های بازی:

	cost	time
L0	74	3ms
L1	76	3ms
L2	121	7ms
L3	1400	3693ms

پیاده سازی الگوریتم BFS :

الگوریتم BFS تمام گره های بالاتر از سطحی ترین راه حل را پردازش می کند با فرض اینکه کم عمق ترین راه حل در عمق S باشد. جست و جو $O(b^S)$ زمان خواهد برد و fringe آن نیز به اندازه $O(b^S)$ حافظه خواهد گرفت چون تقریباً ردیف s ام به همان قدر گره دارد

از صف queue برای fringe و تضمین اینکه اولین راه حل یافت شده کوتاه ترین مسیر از نظر تعداد گام ها هست

BFS اگر راه حلی باشد و عمق جواب محدود باشد کامل هست

و در صورت برابری هزینه ها بهینه نیز هست

BFS که کدش را زده ایم از داده ساختار deque به عنوان صف استفاده می کند برای گره هایی که باید بازدید شوند یا همان fringe استفاده می کند و از یک مجموعه برای نگهداری گره های visit شده استفاده می کنیم. از یک شمارنده expanded_count استفاده میکنیم تا تعداد expand ها در دستمان باشد. در حلقه while تا زمانی که صف خالی نشده است و تعداد گره های expand شده از حد بیشتر نشده گره را از صف خارج میکند بررسی میکند که هدف هست یا نیست بعد آن را با توجه به بن بست ها هرس میکند و حالات بن بست را بررسی نمی کند. گره را بسط می دهد و جانشینان را بررسی میکند که آیا قبلاً visit شده بودند یا نه visit نشده ها را به صف اضافه میکند

کد پیاده سازی BFS :

```
def bfs_fast(problem: SokobanProblem, max_expanded: int = 350000) ->
    Tuple[Optional[List[str]], Dict]:
    start = problem.initial
    # (state, path)
    queue = deque([(start, [])])
    visited = {start}
    expanded_count = 0
```

```

while queue and expanded_count < max_expanded:

    state, path = queue.popleft()
    expanded_count += 1

    if problem.is_goal(state):
        return path, {"expanded": expanded_count}

    if problem.use_deadlocks and problem.is_deadlock(state):
        continue

    for action, next_state, cost in problem.successors(state):

        if next_state not in visited:

            visited.add(next_state)

            queue.append((next_state, path + [action]))

return None, {"expanded": expanded_count}

```

نتایج خروجی برای level های بازی :

	cost	time
L0	5	1ms
L1	12	1ms
L2	27	1ms
L3	96	4417ms

پیاده سازی الگوریتم UCS :

در این الگوریتم تمام گره هایی که هزینه کمتر از ارزان ترین راه حل دارند را پردازش میکند و به فرض محدود بودن هزینه جواب بهینه و حداقل هزینه قوس مثبت باشد کامل است.

و اثبات میشود که جست و جوی هزینه یکنواخت بهینه نیز می باشد

این الگوریتم بر اساس افزایش هزینه گره ها را بررسی میکند و گره هایی که نزدیک تر هستند زودتر بررسی می شوند گزینه ها را در هر جهت بررسی می کند و اطلاعات درمورد هدف را در نظر نمی گیرد و از این لحاظ بد هست

در این الگوریتم از صف اولویت برای fringe استفاده می شود.

در کد ابتدا گره شروع به صف اولویت اضافه میشود. یک دیکشنری برای ذخیره ارزان ترین هزینه برای رسیدن به هر وضعیت با هزینه 0 در نظر میگیریم. بعد یک دیکشنری دیگر برای ردیابی والدین در نظر میگیریم برای بازسازی مسیر . یک مجموعه visited برای جلوگیری از بسط مجدد گره های بسط یافته شده در نظر می گیریم. بعد تا زمانی که صف خالی نشده است

گره با کمترین هزینه را خارج می کنیم و بررسی می کنیم که آیا گره مد نظر هدف هست یا نیست اگر بود path را برمیگردانیم و اگر تعداد گره های بسط داده شده به بالاتر از یک حدی رسید یک پیام چاپ میکنیم. اگر گره ای که از صف خارج شده بن بست باشد نادیده گرفته میشود و حالت بن بست هرس

میشود. گره کنونی به عنوان بسط یافته علامت گذاری می شود در `visited`. سپس گره بسط داده میشود و جانشینان آن بررسی میشوند. روی تمام جانشینان قانونی حالت فعلی تکرار میکند. محاسبه هزینه کل مسیر تا حالت جانشین.

بهروزرسانی مسیر برای بازسازی انجام میدهد و شمارنده تساویها را افزایش میدهد. بررسی میکند که حالت جانشین بنیست نباشد. گره جانشین را به صف اولویت اضافه میکند. اگر صف خالی شود و هدف یافت نشود، جستجو با شکست مواجه میشود.

تابع `reconstruct_path` مسوول بازسازی توالی امسیر پس از یافتن هدف توسط کد `ucs` هست

کد نوشته شده برای UCS :

```
def reconstruct_path(parent: Dict[State, Tuple[Optional[State], Optional[str]]], goal: State) -> List[str]:
    actions: List[str] = []
    s = goal
    while True:
        ps, a = parent.get(s, (None, None))
        if ps is None:
            break
        actions.append(a)
        s = ps
    actions.reverse()
    return actions

def ucs_search(problem: SokobanProblem, max_expanded: int = 500000) -> Tuple[Optional[List[str]], Dict]:
    start = problem.initial

    frontier: List[Tuple[float, int, State]] = []
    counter = 0
    heapq.heappush(frontier, (0.0, counter, start))

    cost_so_far: Dict[State, float] = {start: 0.0}

    parent: Dict[State, Tuple[Optional[State], Optional[str]]] = {start: (None, None)}
```

```
visited: Set[State] = set()
expanded_count = 0

while frontier:

    current_cost, _, state = heapq.heappop(frontier)

    if problem.is_goal(state):
        path = reconstruct_path(parent, state)
        return path, {"expanded": expanded_count}

    if expanded_count >= max_expanded:
        print("جستجو به دلیل رسیدن به محدودیت گره‌های بسط یافته متوقف شد")
        break

    if problem.use_deadlocks and problem.is_deadlock(state):
        continue

    visited.add(state)
    expanded_count += 1

    for action, next_state, step_cost in problem.successors(state):

        new_cost = cost_so_far[state] + step_cost

        if next_state not in cost_so_far or new_cost <
cost_so_far[next_state]
```

```

cost_so_far[next_state] = new_cost
parent[next_state] = (state, action)

counter += 1
heapq.heappush(frontier, (new_cost, counter,
next_state))

return None, {"expanded": expanded_count}

```

نتایج خروجی :

	cost	time
L0	5	1ms
L1	8	1ms
L2	27	6ms
L3	90	5552ms

تابع هیوریستیک مورد استفاده:

مشکل اصلی: فضای حالت (State Space) بازی بسیار بزرگ است و جستجوی ناآگاه (BFS/DFS/UCS) عملاً غیرممکن است یا بسیار کند عمل میکند.

راهحل: تابع هیوریستیک ($h(s)$) برای تخمین هزینه باقیمانده تا رسیدن به هدف استفاده میشود و الگوریتمهای آگاهانه را قادر میسازد تا به طور هوشمندانه، مسیرهای امیدوارکننده را دنبال کند
من از یک تابع هیوریستیک بسیار قوی و چند وجهی استفاده کرده ام

درکد من متغیر total مقدار نهایی هیوریستیک برای آن state را نگهداری میکند و در طول بخش های مختلف تابع هیوریستیک دایم بررسی می شود و ابدیت میشود در واقع همان جمع کل هزینه های تخمین ی می باشد

اجزای کلیدی تابع هیوریستیک من :

اجزای اصلی تابع هیوریستیک:

1) فاصله جعبه ها تا اهداف : مجموع فاصله منتهن هر جعبه تا نزدیک ترین هدف ازاد که ضریب 2 اهمیت این پارامتر را افزایش می دهد و درواقع همان بخش اصلی تخمین ما هست

2) فاصله بازیکن تا جعبه : تشویق الگوریتم به حرکت به سمت جعبه ای که باید هل داده شود دراین جا هم از فاصله منتهن استفاده کرده ایم و اینجا به greedy کمک کرده ایم بازیکن را به سمتی هدایت کند که سریع تر بتواند جعبه ها را هل دهد .

هوشمندی و مدیریت اثر سیب و سم :

3) پاداش جعبه روی هدف: پاداش بزرگ : ما به ازای تعداد جعبه های روی هدف 15 امتیاز جایزه میدهم و از هیوریستیک مان کم می کنیم

4) اگر تایمر سیب فعال باشد 10 واحد پاداش داده میشود .و اگر جعبه های برای هل دادند موجود باشد باز 5 واحد پاداش داده میشود

5) اگر اثر سم فعال بود 20 واحد جریمه در نظر گرفته می شود

6) جریمه بن بست احتمالی: اگر به بن بست قطعی همان گوشه ها برسد 100 واحد جریمه میشود ولی نه اگر که به بن بست نسبی برسد یعنی بین دو دیوار موازی گیر بکند 30 واحد جریمه می شود بن بست لبه هم جریمه 15 واحدی دارد.
از طرفی نزدیکی به سم خطرناک هست و برای آن هم جریمه 15 واحدی در نظر گرفته شده است

7) در نهایت به ازای هر سیب که جمع اوری کرده است 8 برابرش پاداش دریافت می کند

در نهایت کد ما max بین 0 و total را برگرداند که علتش ترس از منفی شدن total در برخی مسایل هست و جلوگیری از اعلام هیوریستیک منفی هست

مزایای این هیوریستیک

- 1) ترکیبی بودن: هیوریستیک من ترکیبی از جنبه های مکانی و مفهومی هست
- 2) عدم اتکا به فاکتور های ساده
- 3) جلوگیری از وقوع بن بست ها و عدم بررسی ان ها
- 4) سریع بودن و هوشمندانه بودن
- 5) تاثیر گذاری مستقیم قدرت های موقت سیب و سم روی تصمیم گیری ها
- 6) منطقی بودن و متناسب بودن میزان جریمه و پاداش ها

معایب این هیوریستیک

1) تابع هیوریستیک admissible نیست : تضمین نمیکند که همیشه کمتر از هزینه واقعی تخمین بزند. چون ما خودمان دلخواه ضرابی را به پاداش و جریمه ها داده ایم و این لزوما با هزینه های واقعی سازگار نیست. اما چون برای greedy ما نیازی به admissibility نداریم برای greedy هیوریستیک خوبی به نظر میرسد

پیاده سازی این تابع هیوریستیک:

```
def manhattan_distance(a: Tuple[int, int], b: Tuple[int, int]) -> int:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def heuristic_intelligent(state: State, problem: SokobanProblem) -> float:
    """
    با در نظر گیری Sokoban هیوریستیک هوشمند برای:
    - فاصله جعبه ها تا اهداف
    - موقعیت بازیکن
    - اثر سیب و سم
    - الگوهای بن بست
    - پیشرفت بازی
    """
    total = 0.0
```

```

# 1. هسته اصلی (فاصله جعبه‌ها تا اهداف)

for box in state.bboxes:

    min_box_dist = float('inf')

    for goal in problem.goals: # استفاده میکنیم از problem

        dist = manhattan_distance(box, goal)

        min_box_dist = min(min_box_dist, dist)

    total += min_box_dist * 2 # چون مهمترین بخش است 2 ضرب

```

فاصله بازیکن تا نزدیکترین جعبه

```

# 2.

if state.bboxes:

    player_to_box = float('inf')

    for box in state.bboxes:

        dist = manhattan_distance(state.player, box)

        player_to_box = min(player_to_box, dist)

    total += player_to_box

```

پاداش برای جعبه‌های روی هدف

```

# 3.

boxes_on_goal = len(state.bboxes & problem.goals)

total -= boxes_on_goal * 15 # پاداش زیاد برای پیشرفت

```

پاداش برای قدرت موقت (اثر سیب)

```

# 4.

if state.apple_timer > 0:

    total -= 10 # پاداش برای داشتن اثر سیب

    # پاداش بیشتر اگر جعبه‌هایی برای هل دادن وجود دارد

    if len(state.bboxes - problem.goals) > 0:

        total -= 5

```

جریمه برای وضعیت بد (اثر سم)

```

# 5.

if state.poison_active:

    total += 20 # جریمه برای سم فعال

```

جریمه برای الگوهای بنبست احتمالی. 6.

```
deadlock_risk = 0

for box in state.bboxes:
    if box not in problem.goals:
        # بنبست قطعی
        if problem._is_simple_corner_deadlock(box):
            deadlock_risk += 100

        # بنبست نسبی
        elif problem._is_wall_deadlock(box, state.bboxes):
            deadlock_risk += 30

        # بنبست لبه
        elif problem._is_edge_deadlock(box):
            deadlock_risk += 15

        for poison in state.poisons:
            if manhattan_distance(box, poison) <= 1:
                deadlock_risk += 15

total += deadlock_risk

apples_collected = len(problem.apples_initial) - len(state.apples)
total -= apples_collected * 8

return max(0, total)
```


پیاده سازی الگوریتم Greedy :

الگوریتم جستجوی حریصانه یک استراتژی جستجوی آگاهانه (Informed Search) است که هدف آن یافتن سریعترین مسیر به هدف است. این الگوریتم تنها بر اساس تخمین هزینه باقیمانده (هیوریستیک) تصمیمگیری میکند و هزینه واقعی مسیر طی شده را نادیده میگیرد. به همین دلیل، به آن "حریصانه" گفته میشود؛ زیرا در هر مرحله، کوتاهترین مسیر تخمینی را بدون فکر کردن به آینده انتخاب میکند.

الگوریتم حریصانه از یک صف اولویتدار استفاده میکند تا همواره گره‌های را کاوش کند که نزدیکترین تخمین را به هدف دارد (کمترین مقدار هیوریستیک).

چند ویژگی که الگوریتم Greedy در خود دارد:

- 1 وابستگی مطلق به هیوریستیک : موفقیت این الگوریتم کاملاً به کیفیت تابع heuristic_intelligent بستگی دارد. اگر هیوریستیک تخمین بدی داشته باشد الگوریتم ممکن است در مسیرهای بن‌بست یا طولانی گیر کند.
 - 2 عدم بهینگی: greedy search تضمین نمیکند که کوتاه‌ترین مسیر از نظر گام یا ارزان‌ترین مسیر از نظر هزینه را پیدا بکند این بزرگ‌ترین ضعف آن نسبت به BFS و UCS است
 - 3 مزیت سرعت: مزیت اصلی greedy سرعت بالای آن در فضای حالت بزرگ است زیرا با انتخاب حریصانه از کاوش بخش وسیعی از فضای جست و جو که به نظر دور می‌آیند اجتناب می‌کند
- الگوریتم حریصانه برای یافتن سریع یک راه حل خوب در زمان محدود انتخاب مناسبی است اما برای یافتن بهترین راه حل باید از A^* استفاده کرد

پیاده سازی کد:

```
def greedy_search(problem: SokobanProblem,
                 heuristic_func=heuristic_intelligent):
    start = problem.initial
    frontier = []
```

```
counter = 0

start_priority = heuristic_func(start, problem)

heapq.heappush(frontier, (start_priority, counter, start, []))

visited = set()

while frontier:

    _, _, state, path = heapq.heappop(frontier)

    if problem.is_goal(state):
        return path, {"expanded": len(visited)}

    if state in visited:
        continue

    if problem.use_deadlocks and problem.is_deadlock(state):
        continue

    visited.add(state)

    for action, next_state, cost in problem.successors(state):

        priority = heuristic_func(next_state, problem)
        counter += 1

        if not (problem.use_deadlocks and
problem.is_deadlock(next_state)):
            heapq.heappush(frontier, (priority, counter,
next_state, path + [action]))
```

```
return None, {"expanded": len(visited)}
```

نتایج خروجی:

	cost	time
L0	5	1ms
L1	12	1ms
L2	30	4ms
L3	114	87ms

پیاده سازی الگوریتم A^* :

A^* الگوریتمی هست که باید هم هزینه ها را لحاظ کند و هم هیوریستیک را در نظر بگیرد. و ترکیبی است از UCS و Greedy. در UCS معیار گسترش در لبه $g(n)$ بود. در Greedy معیار گسترش در لبه $h(n)$ بود و در A^* معیار گسترش در لبه $h(n) + g(n)$ میباشد. میدانیم که A^* با هر هیوریستیکی لزوماً بهینه نیست و نیازمند هیوریستیک Admissible هست که مفهومی قابل قبول بودن یعنی اینکه هیوریستیک همواره کمتر مساوی هزینه واقعی را برآورد بکند. در جست و جوی گرافی علاوه بر شرط قابل قبول بودن شرط دیگری نیز وجود دارد و آن اینکه هیوریستیک ما باید حتماً سازگار نیز باشد بدین معنا که اختلاف هیوریستیک های مجاور حتماً کمتر مساوی هزینه واقعی یال باشد.

برای A^* از صف با اولویت استفاده شده است. و از همان هیوریستیک heuristic_intelligent استفاده شده است
توضیحات کد:

حالت اولیه ی بازی (جایی که بازیکن و جعبه ها در ابتدا هستند) گرفته می شود
صف همان لیست اولویت دار را تشکیل می دهیم. حالت های در حال بررسی در آن نگهداری میشوند.
برای اینکه دو حالت با اولویت مساوی را بتوانیم در heap مقایسه بکنیم از شمارنده ای استفاده می کنیم که ترتیب ورود را حفظ کند .

بعد هزینه طی شده از ابتدای مسیر تا الان را $g(n)$ را صفر قرار می دهیم

مقدار تابع اولویت $f(n)$ برای حالت اولیه محاسبه می شود: $h(n) + g(n) = f(n)$

حالت اولیه در صف اولویت قرار می گیرد . هر عنصر شامل اطلاعات زیر هست :

(اولویت و شمارنده و هزینه ی طی شده و حالت فعلی و مسیر طی شده)

دیکشنری برای نگهداری بهترین هزینه ای که تاکنون برای هر حالت پیدا شده $state \rightarrow g(n)$

مجموعه ای برای نگهداری حالت هایی گسترش داده شده اند تا دوباره پردازش نشوند

حلقه اصلی:

تا زمانی که صف خالی نشده الگوریتم ادامه می دهد.

حالتی با کمترین $f(n)$ از heap خارج می شود یعنی همان حالتی که فعلا امیدبخش ترین مسیر هست
اگر حالت فعلی هدف باشد (یعنی همه جعبه ها روی اهداف قرار گرفته اند) مسیر بازگشت داده میشود

Expanded هم تعداد حالت هایی گسترش داده شده را گزارش می کند

اگر این حالت قبلا بررسی شده ردش می کند تا دوباره گسترش داده نشود

اگر سیستم تشخیص بن بست فعال است و حالت فعلی بن بست است.

ان را نادیده می گیرد تا زمان تلف نشود

این حالت به عنوان بازدید شده علامت می خورد

همه حالت های فرزند یعنی تمام حرکات ممکن از وضعیت فعلی تولید می شوند

هزینه واقعی ($g(n)$) برای حالت جدید محاسبه می شود معمولا $cost = 1$

بعد بررسی می کنیم آیا این مسیر بهتر از مسیر های قبلی است یا نه . اگر این حالت اولین بار است با هزینه
نه جدید کمتر است ادامه بده

مقدار جدید $g(n)$ ثبت می شود سپس $f(n) = g(n) + h(n)$ برای حالت جدید محاسبه می شود

شمارنده یکی زیاد می شود تا ترتیب در heap محفوظ بماند

اگر حالت جدید بن بست نیست ان را با اولویت جدید در صف قرار می دهد

مسیر طی شده نیز به روز رسانی می شود

چنانچه صف خالی شد و به هدف نرسید یعنی هیچ مسیر ممکن برای رسیدن به هدف وجود ندارد

نتایج کد:

	cost	time
L0	5	1ms
L1	8	9ms
L2	27	12ms
L3	98	1213ms

